RAPPORT D'INGÉNIERIE DES DONNÉES · PROJET 4

Une panne détectée en quelques secondes

Supervision temps réel de bornes de recharge :
Kafka, Spark Structured Streaming, Iceberg, Grafana et Terraform

LUMU BAMANA MANASSE

Data engineer

Date	Octobre 2026
Version	1.0.0 (livrée et recettée)
Client	Lumio Recharge (entreprise fictive) · Mission LUM-2026-002
Dépôt GitHub	https://github.com/MediJr/ev-charging-realtime-streaming
Langue	Français · *English version available in the repository*

Scénario fictif, bornes simulées, données carbone réelles : l'entreprise cliente et ses interlocuteurs sont inventés pour simuler une mission professionnelle complète ; les mesures des bornes sont produites par un simulateur. L'intensité carbone est la donnée éCO2mix temps réel de RTE (Open Data Réseaux Énergies, Licence Ouverte). Ce projet n'est ni commandé, ni validé, ni affilié à RTE.

Fiche du document

Projet	Projet 4 du portfolio : streaming, lakehouse et infrastructure en code
Mission	LUM-2026-002 : « Savoir en quelques secondes qu'une borne ne va pas bien »
Client	Lumio Recharge, opérateur de 1 200 bornes (fictif), à l'attention de Sophie Bernard et Karim Haddad
Auteur	Lumu Bamana Manasse, data engineer
Période de réalisation	Octobre 2026
Données	200 bornes simulées (une mesure toutes les 5 s) ; intensité carbone réelle de RTE
Statut	LIVRÉ RECETTÉ
Code source	https://github.com/MediJr/ev-charging-realtime-streaming

Historique des versions

Version	Date	Objet
0.1	26/10/2026	Cadrage : brief, compte rendu, cahier des charges
0.5	26/10/2026	Plateforme de streaming, lakehouse, supervision, Terraform
0.9	27/10/2026	Endurance de 31 h : deux défauts corrigés
1.0.0	27/10/2026	Recette (panne, charge, rattrapage), rapport final (français et anglais)

Guide de lecture

Direction des opérations	1. Résumé exécutif et 10. Résultats
DSI (passage au cloud)	4. Architecture, 8. Infrastructure GCP, 12. Feuille de route
Ingénieur reprenant le projet	5. Traitement, 6. Lakehouse, 9. Recette, 11. Retour d'expérience

Table des matières

Fiche du document 2

1 Résumé exécutif 4

2 Contexte et cadrage 5

3 Données **5**

4 Architecture **6**

5 Traitement Spark **6**
5.1 Temps d'événement, watermark et dédoublonnage 7
5.2 L'état par borne et la détection d'une absence 7

6 Lakehouse Iceberg **7**

7 Supervision **8**

8 Infrastructure Google Cloud en Terraform **8**

9 Recette **8**
9.1 Rapprochement : chaque message est expliqué 9

10 Résultats **9**

11 Retour d'expérience **12**

12 Feuille de route **12**

1 Résumé exécutif

Le besoin. Lumio Recharge apprend qu'une borne est en panne quand un client appelle, parfois des heures plus tard. Ses bornes envoient pourtant leur puissance toutes les 5 secondes. L'exploitation veut être alertée **en moins de 30 secondes**, voir le **CO_2 de chaque recharge en direct**, et superviser le parc sur un écran. La DSI prévoit Google Cloud l'an prochain.

La solution. Une plateforme de **streaming** démarrée par un seul `docker compose up`. Les mesures passent par un journal d'événements compatible **Kafka** (Redpanda), sont traitées en **temps d'événement** par **Spark Structured Streaming** (watermark, dédoublonnage, état par borne), conservées dans un **lakehouse Apache Iceberg**, enrichies de l'intensité carbone **réelle** de RTE, et affichées dans **Grafana**. L'infrastructure Google Cloud cible est décrite en **Terraform**.

27 / 27 anomalies détectées	**11,6 s** délai de détection au 95e centile	**0** fausse alerte	**0** message perdu ou doublé après arrêt brutal
83 292 messages rapprochés un par un	**8,4 s** pour rattraper 3 654 messages	**≈ 1 440** messages/s (capacité mesurée)	**22** tests, dont 5 Spark de bout en bout

Ce qu'il faut retenir

- **Q1 · Délai :** les 27 anomalies injectées sont détectées en 5,9 s (médiane) et 11,6 s (95e centile). Objectif : 30 s.
- **Q2 · Anomalies :** borne muette, surpuissance, borne bloquée, chacune ouverte puis résolue, sans déluge d'alertes.
- **Q3 · CO_2 :** 186 recharges, 4,8 MWh, 211 kg de CO_2 à l'intensité RTE réelle de la soirée (44 g/kWh).
- **Q4 · Robustesse :** Spark tué en plein flux, reprise sans perte ni doublon. Capacité d'environ 1 440 messages/s, soit quelque 7 000 bornes : six fois le parc actuel.

Trois défauts ont été trouvés en faisant tourner la plateforme pour de vrai (31 h d'endurance, panne, surcharge), corrigés et couverts par des tests : une colonne de dates vide refusée par Arrow, des identifiants de mesure répétés après un redémarrage, et des bornes muettes manquées pendant un rattrapage.

2 Contexte et cadrage

Demande du client	Conséquence technique
« Détecter en quelques secondes »	Traitement **continu** (streaming), pas par lots
« Une borne qui ne répond plus »	Détecter une **absence** d'événement : état par borne et délai
Réseau 4G qui retarde ou double les messages	**Temps d'événement**, watermark, dédoublonnage
« Sans tout réécrire sur Google Cloud »	Formats ouverts (Iceberg), API Kafka, infrastructure en **Terraform**
Une panne ne doit rien perdre ni doubler	Points de reprise (checkpoints), écritures idempotentes

Table 1 – De la demande aux exigences techniques

Anomalie	Règle (cadrage)	Gravité
Borne muette	Aucun message depuis plus de 60 s	critique
Surpuissance	Puissance supérieure à 110 % de la puissance nominale	haute
Borne bloquée	Session ouverte à 0 kW depuis plus de 2 minutes	moyenne

Table 2 – Les trois anomalies et leurs règles

3 Données

Mesures des bornes (simulées). 200 bornes sur 12 sites, deux tiers en 22 kW et un tiers en 50 kW, une mesure toutes les 5 s : puissance, énergie de la session, état, session en cours. Le simulateur injecte **volontairement** des défauts, et publie chaque injection dans un topic de **vérité terrain** (`sim.injections`) qui permet de vérifier chaque détection.

Défaut injecté	Fréquence	Ce qu'il éprouve
Borne muette, surpuissance, borne bloquée	≈ 0,3 par borne et par heure	Les trois règles d'anomalie
Message en double	1 %	Dédoublonnage
Retard de 5 à 40 s	1 %	Temps d'événement
Retard de 3 à 5 min	0,2 %	Watermark : écarté et compté

Table 3 – Défauts injectés par le simulateur (ADR 005)

Intensité carbone (réelle). Un collecteur lit toutes les 5 minutes l'API éCO2mix temps réel de RTE, la même source qu'au projet 2, et publie chaque créneau de 15 minutes dans un topic **compacté** : seule la dernière valeur de chaque créneau est conservée.

4 Architecture

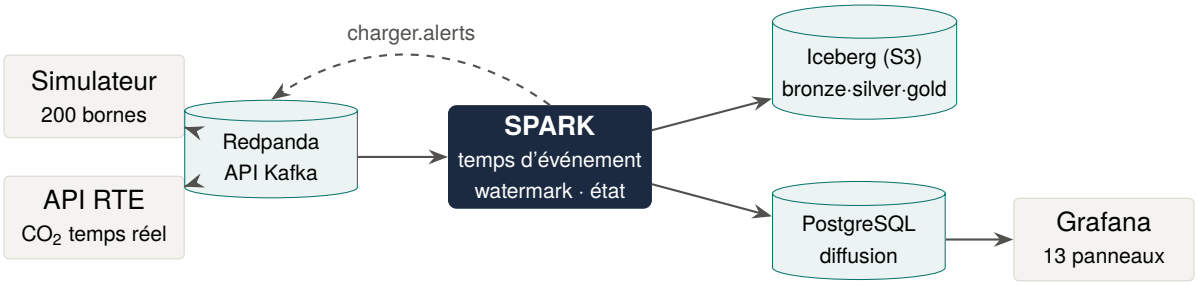


Figure 1 – De la borne à l'écran de supervision

Rôle	Choix	Pourquoi (ADR)
Journal d'événements	Redpanda v25.3	Protocole Kafka, un seul binaire, léger ; remplaçable par un Kafka managé (001)
Traitement	Spark 4.0 Structured Streaming	Temps d'événement, watermark, état tolérant aux pannes (001)
Lakehouse	Iceberg 1.10 sur S3, catalogue JDBC	Transactions, format ouvert lu par BigQuery (002)
Diffusion	PostgreSQL 17	Rafraîchissement de l'écran en quelques millisecondes (003)
Supervision	Grafana 12, provisionné en code	Standard de l'observabilité (003)
Cloud	Terraform, fournisseur Google 8.8	Infrastructure relue et versionnée avant d'exister (004)

Table 4 – Choix techniques

5 Traitement Spark

Une seule application Spark exécute six requêtes, chacune avec son point de reprise :

Requête	Rythme	Traitement	Destination
bronze	30 s	Aucun : le message tel que reçu, avec sa position Kafka	Iceberg
silver	30 s	Lecture du JSON, watermark de 2 min, dédoublonnage	Iceberg
monitor	5 s	État par borne et règles d'anomalie	PostgreSQL, Kafka, Iceberg
minutes	30 s	Énergie par borne et par minute close, CO_2	PostgreSQL, Iceberg
grid_co2	30 s	Intensité carbone RTE	PostgreSQL, Iceberg
injections	30 s	Vérité terrain du simulateur	PostgreSQL

Table 5 – Les six requêtes en continu

5.1 Temps d'événement, watermark et dédoublonnage

Une mesure est rangée selon l'heure **où la borne l'a produite**, pas selon l'heure où elle arrive. La **watermark** de 2 minutes indique jusqu'où Spark attend les retardataires : au-delà, une mesure est écartée, et comptée dans le suivi des micro-lots. Le dédoublonnage porte sur l'identifiant de la mesure, `event_id`.

5.2 L'état par borne et la détection d'une absence

Les règles sont écrites en **Python pur** (`lumio_stream.monitor`), testées sans Spark. Spark les appelle pour chaque borne avec ses nouvelles mesures et l'état mémorisé (`applyInPandasWithState`). Une borne qui n'envoie plus rien n'appelle plus rien : c'est un **délai de traitement** de 60 s, posé à chaque mesure reçue, qui déclenche l'alerte « borne muette ».

> **Limite découverte en recette : le rattrapage**
>
> Quand Spark rattrape un retard, il traverse plusieurs minutes de temps d'événement en quelques secondes : le délai de traitement ne se déclenche pas, et 3 bornes muettes ont été manquées. Un **filet de sécurité en temps d'événement** a été ajouté : un trou de plus de 60 s entre deux mesures, constaté au retour de la borne, ouvre et ferme l'alerte. Elle arrive en retard, mais n'est **jamais perdue**.

6 Lakehouse Iceberg

Table	Grain	Contenu
bronze.telemetry_raw	un message Kafka	Message brut, partition et position Kafka
silver.telemetry	une mesure	Champs typés, dédoublonnés
gold.charger_minute	borne × minute	Énergie, CO_2
gold.alerts	changement d'état	Alerte ouverte ou résolue, heures d'anomalie et de détection

TABLE 6 – Les tables du lakehouse (partitionnées par jour)

Exactement une fois. Le puits Iceberg natif de Spark l'est par construction. Pour les écritures faites dans `foreachBatch`, qui ne garantit qu'« au moins une fois », chaque ajout est marqué du numéro de micro-lot dans l'instantané Iceberg : un lot rejoué après une panne est ignoré. Les écritures de diffusion sont des UPSERT.

Maintenance. Le streaming écrit un petit fichier par micro-lot. Un script quotidien supprime les mesures brutes de plus de 90 jours, compacte les petits fichiers et expire les anciens instantanés.

7 Supervision

L'écran Grafana (13 panneaux, provisionnés en code, accès anonyme en lecture seule) montre en premier les **alertes ouvertes** triées par gravité, puis les bornes en ligne, la puissance du parc, l'intensité RTE, le délai de détection au 95e centile, le débit, les recharges en cours avec leur CO_2, et la santé du flux (lignes écartées par la watermark). Un `StreamingQueryListener` enregistre chaque micro-lot dans la base de diffusion.

8 Infrastructure Google Cloud en Terraform

Ressource	Choix
Bucket du lakehouse	Région Paris, accès public interdit, versionnage, mesures brutes supprimées après 90 jours
BigQuery + BigLake	Dataset des tables gold ; connexion BigLake pour lire Iceberg sans copie
Comptes de service	Un par usage : le traitement écrit, la supervision lit seulement
Alerte de budget	50, 90 et 100 % d'un budget mensuel

Table 7 – Infrastructure décrite (ADR 004)

`terraform fmt`, `init -backend=false` et `validate` passent, en local et dans l'intégration continue. Le `plan` demande des identifiants GCP : il sera fait avec la DSI. Correspondance : SeaweedFS devient Cloud Storage, Spark local devient Dataproc Serverless (même code), Redpanda devient Kafka managé (même protocole).

9 Recette

La recette a été passée sur une **plateforme vierge**, avec un arrêt brutal de Spark en plein flux, puis un test de charge.

#	Critère	Résultat
C1	Démarrage avec `docker compose up` sur une plateforme vierge	✓
C2	Les trois anomalies injectées détectées (27 / 27, 0 fausse alerte)	✓
C3	Délai de détection < 30 s au 95e centile (11,6 s)	✓
C4	Doublons comptés une fois, retardataires comptés à part	✓
C5	Arrêt brutal de Spark : ni perte, ni doublon	✓
C6	CO_2 calculé avec l'intensité RTE réelle	✓
C7	Terraform validé, tests verts	✓
C8	Aucun secret versionné	✓

Table 8 – Critères d'acceptation : 8 sur 8

9.1 Rapprochement : chaque message est expliqué

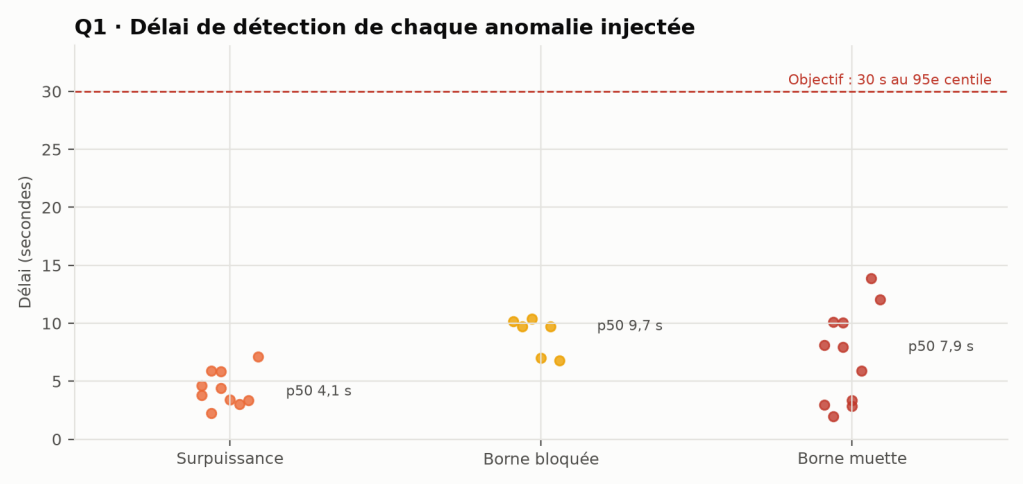
spark/recette.py · messages arrivés avant 19 :46 :59

```
messages bronze                :    83,292   (positions Kafka distinctes : 83,292)
identifiants distincts         :    82,484   -> doublons écartés : 808
mesures silver                 :    82,347   (identifiants distincts : 82,347)
identifiants absents de silver :       137
équation : 83,292 = 82,347 + 808 + 137  -> OK
message lu deux fois par Spark : 0
injections avant la coupure    : {'too_late': 164, 'late': 882, 'duplicate': 808}
absents de silver expliqués par un retard injecté de 3 à 5 min : 137 / 137
```

Les 808 doublons écartés sont exactement les 808 doublons injectés. Les 137 mesures absentes sont toutes des retards injectés au-delà de la watermark. Aucun message Kafka n'a été lu deux fois, malgré l'arrêt brutal de Spark : le premier micro-lot après la relance porte le numéro 41, juste après le 40 validé avant la panne, et rattrape 3 654 messages en 8,4 s.

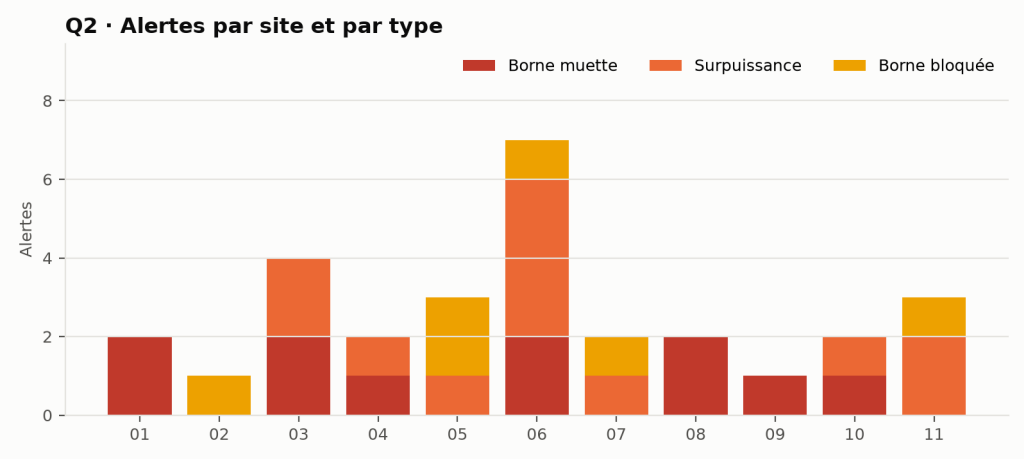
10 Résultats

Q1 · Délai de détection

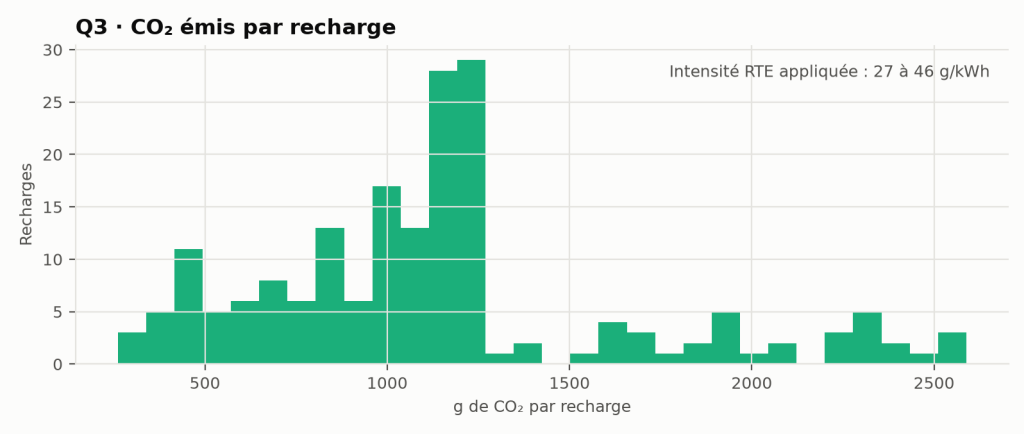


Les 27 anomalies injectées pendant la période nominale sont toutes détectées, sans fausse alerte : surpuissance en 4,1 s (médiane), borne bloquée en 9,7 s, borne muette en 7,9 s. Le 95e centile est de **11,6 s**, le maximum de 13,9 s.

Q2 · Alertes par site

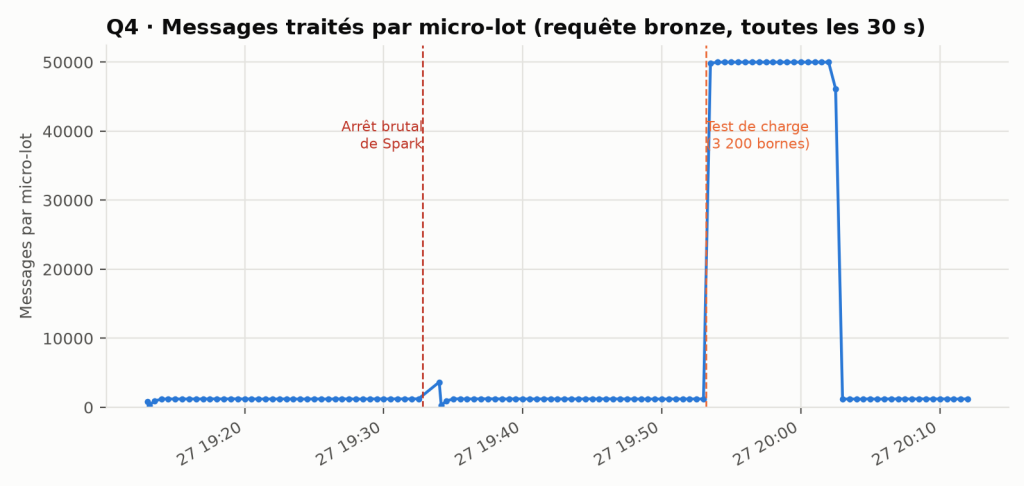


Q3 · CO_2 des recharges



186 recharges ont délivré 4,8 MWh et émis 211 kg de CO_2, à l'intensité réelle du réseau ce soir-là (44 g/kWh en moyenne) : 1,1 kg par recharge en médiane. Le même calcul est affiché en direct pour chaque recharge en cours.

Q4 · Panne, rattrapage et charge



Test de charge (≈ 2 960 messages/s)	**Résultat**
Bronze et silver	Suivent : jusqu'à 5 700 et 4 600 messages/s
Moniteur à état	**Goulot d'étranglement** : ≈ 1 440 messages/s ; détection à 130-160 s pendant la surcharge
Stabilité	Aucun plantage ; retard résorbé 10 minutes après la fin de la surcharge
Capacité	≈ 7 000 bornes à une mesure toutes les 5 s, pour un parc de 1 200

TABLE 9 – Ce que la plateforme tient sur un poste

11 Retour d'expérience

Défaut ou incident	Leçon
Les tests Spark étaient ignorés sans le dire	Un test « ignoré » n'est pas un test réussi : vérifier le nombre de tests exécutés
Colonne de dates vide refusée par Arrow	Le premier lot réel contient des cas qu'aucun exemple ne contenait
`event_id` répétés après un redémarrage	Une clé de dédoublonnage doit être unique **dans le temps**, pas dans une exécution
Rapprochement faux au premier essai	Deux tables écrites par deux requêtes ne sont jamais au même instant : comparer un ensemble fixe
Bornes muettes manquées au rattrapage	Un délai en temps de traitement n'a pas de sens quand le temps est « accéléré »
Arrêts de Spark après mise en veille	Les checkpoints et la politique de redémarrage ont repris seuls ; un `docker kill` manuel, non

TABLE 10 – Ce que ce projet a appris

12 Feuille de route

1. **Capacité** : moniteur en Scala (`transformWithState`) ou cluster Dataproc, plus de partitions Kafka.
2. **Alertes sortantes** : consommateur de `charger.alerts` vers SMS et ticketing, avec dédoublonnage.
3. **Déploiement GCP** : `terraform plan` avec la DSI, puis Dataproc Serverless et Kafka managé.
4. **Vraies bornes** : passerelle OCPP vers le topic `charger.telemetry`, sans changer le traitement.